Дискретная математика

Теория типов

Константин Чухарев

Осень 2026

Простые типы

> *Есть два способа построить программу: настолько простую, что недостатков очевидно нет, или настолько сложную, что очевидных недостатков нет.*
>
> **— Тони Хоар**

Типы

Компилятор не даёт сложить число и строку — ловит ошибку до запуска. Типы формализуют, какие термы можно составлять.

Определение 1 (Простые типы)

- Базовые: Nat, Bool.
- Функциональный: $\sigma \to \tau$.

Никаких других типов нет.

Тип — классификация терма по поведению.

Тип — не значение, а инвариант: он фиксирует, что с термом можно делать.

Примеры типов

Тип получает смысл от термов, которые его имеют. Функциональный тип обещает: подайте терм входного типа — получите терм выходного. Насельники трёх форм ниже идут от булевых функций к функциям высшего порядка.

Пример

- $\mathrm{Bool} \to \mathrm{Bool}$ населяют отрицание $\lambda x : \mathrm{Bool}\ .\neg x$ и тождественная функция.
- $\mathrm{Nat} \to \mathrm{Nat} \to \mathrm{Nat}$ — тип сложения: функция двух аргументов, принятая по одному.
- $(\mathrm{Nat} \to \mathrm{Nat}) \to \mathrm{Nat}$ населяет «дважды»: $\lambda s : \mathrm{Nat} \to \mathrm{Nat}\ .s(s0)$.

«Дважды» принимает функцию и возвращает число: применяем s к нулю два раза — выходит число 2.

Параллель с логикой

Тип $\sigma \to \tau$ структурно совпадает с импликацией. «Дайте мне σ — верну τ» — то же самое, что «если σ, то τ».

Функция — конструктивное доказательство импликации: из входных данных она строит результат.

Совпадение стрелки типа и импликации не случайно: изоморфизм Карри–Ховарда будет разобран ниже.

Правила типизации

Три правила

Γ — контекст: набор допущений вида $x : \sigma$.

$\Gamma \vdash M : \sigma$ читается как «из Γ выводится, что M имеет тип σ».

$\Gamma, x : \sigma$ — контекст Γ, расширенный новым допущением $x : \sigma$.

Определение 2 (Правила $\lambda \to$)

- **var**: если $x : \sigma \in \Gamma$, то $\Gamma \vdash x : \sigma$.
- **app**: из $M : \sigma \to \tau$ и $N : \sigma$ получаем $MN : \tau$.
- **abs**: из $\Gamma, x : \sigma \vdash M : \tau$ выводим $\Gamma \vdash \lambda x : \sigma.M : \sigma \to \tau$.

Каждое правило — шаг логического вывода.

Терм корректен, если есть дерево вывода.

Примеры типизации

Пример

- $\lambda x : \mathrm{Nat}\ .x$ имеет тип $\mathrm{Nat} \to \mathrm{Nat}$.
- $\lambda x : \mathrm{Nat}\ .\lambda y : \mathrm{Bool}\ .x$ имеет тип $\mathrm{Nat} \to \mathrm{Bool} \to \mathrm{Nat}$.

Второй аргумент игнорируется — тип это фиксирует.

Проверка типов — алгоритмический поиск дерева.

Для $\lambda \to$ проверка типа занимает линейное время.

Типизируемость

Что типизируется?

Какие термы пройдут проверку типов? Достаточно разобрать два комбинатора: K сохраняет первый аргумент и отбрасывает второй, S подставляет z сразу в два места приложения. Их типы — это схемы аксиом минимальной импликативной логики: всё, что в ней выводимо, типизируемо.

- $K = \lambda x.\lambda y.x$: тип $\sigma \to \tau \to \sigma$.
- $S = \lambda x.\lambda y.\lambda z.xz(yz)$: тип $(\alpha \to \beta \to \gamma) \to (\alpha \to \beta) \to \alpha \to \gamma$.

Что не типизируется?

$Y = \lambda f.(\lambda x.f(xx))(\lambda x.f(xx))$ не типизируется.

Для xx переменная x должна иметь сразу тип $\alpha \to \beta$ и α, что даёт $\alpha = \alpha \to \beta$. Конечный тип не может быть собственным подвыражением себя.

Рекурсивных термов в $\lambda \to$ нет.

Поэтому просто типизированное исчисление не Тьюринг-полно.

Это цена за гарантию остановки.

Сильная нормализация

Теорема 1 (Сильная нормализация)

Каждый *типизируемый* терм $\lambda \to$ редуцируется к нормальной форме.

Порядок редукции не важен.

Доказательство

Доказательство — индукция по типам, методом приводимости Тейта.

Каждому типу σ сопоставляется множество приводимых термов. Для базового типа приводимы (после подстановки) термы, редуцирующиеся к значению. Терм типа $\sigma \to \tau$ приводим, если его результат при любом приводимом аргументе приводим в τ.

Приводимость и индукция

Доказательство

Приводимость держится на двух леммах. Первая: каждый терм типа σ редуцируется к нормальной форме. Вторая: если все свободные переменные терма типа σ приводимы, то приводим и сам терм. Нейтральные термы приводятся, как только приводятся их свободные переменные.

Затем индукцией по выводу типизации показывается, что всякий типизируемый терм приводим в своём типе, и потому нормализуется.

Сохранение типа

Типизируемые термы всегда завершаются.

Проверка типов — гарантия остановки.

Редукция тип не меняет: если $M : \sigma$ и $M \overset{\beta}{\to} M'$, то $M' : \sigma$.

Тип как инвариант вычисления

Пример

$(\lambda x : \mathrm{Nat}\ .\lambda y : \mathrm{Bool}\ .x)3\ \mathrm{true} \xrightarrow{\beta} (\lambda y : \mathrm{Bool}\ .3)\ \mathrm{true} \xrightarrow{\beta} 3.$

Два шага — и цепочка обрывается: дальше редуцировать нечего, перед нами нормальная форма.

На каждом шаге весь терм имеет тип Nat: редукция тип не трогает.

Значение может меняться, тип — нет.

Тип — статическая характеристика программы, верная на каждом шаге.

Карри–Ховард

Соответствие

Теория типов	Логика
тип σ	высказывание
терм $M : \sigma$	доказательство
$\sigma \to \tau$	импликация
абстракция	$\to$-введение
аппликация	modus ponens

Карри–Ховард — изоморфизм между просто типизированным λ-исчислением и интуиционистской логикой высказываний.

Не метафора: каждая строка — точное соответствие.

Термы как доказательства

Пример

- $\lambda x : A.x$ — доказательство $A \to A$.
- $\lambda x : A.\lambda y : B.x$ — аксиома $A \to B \to A$.

Композиция — транзитивность импликации:

$\lambda f : A \to B.\lambda g : B \to C.\lambda x : A.g(fx)$.

Терм и доказательство выражают одно и то же разными словами, но с одной структурой.

Два направления

- Доказываете теорему — терм и есть программа.
- Пишете типизируемую программу — тип и есть теорема.
- Компилятор — верификатор доказательства.

Интуиционистская логика требует конструктивных доказательств.

Три линии сошлись в одном изоморфизме:

- Вычислительная (Чёрч).
- Логическая (Гейтинг).
- Инженерная (де Брёйн).

За пределы $\lambda \rightarrow$

Расширения соответствия

Карри–Ховард не ограничен импликацией.

Каждая новая конструкция типов — новая логическая связка.

Теория типов	**Логика**
тип-произведение $\sigma \times \tau$	конъюнкция $\sigma \wedge \tau$
тип-сумма $\sigma + \tau$	дизъюнкция $\sigma \vee \tau$
пустой тип $\bot$	ложь, ex falso quodlibet

Пара населяет $\sigma \times \tau$ — значит, есть и σ, и τ.

Из $\bot$ выводится любое утверждение — принцип ex falso.

λ-куб

Три вида зависимости между термами и типами — три оси куба Барендрегта.

Определение 3 (Три оси)

- $\lambda\to$: термы зависят от термов.
- $\lambda 2$: термы зависят от типов — полиморфизм.
- λP: типы зависят от термов — зависимые типы.
- $\lambda\omega$: типы зависят от типов — операторы над типами.

Пример

Оператор над типами: $\mathrm{List} : \star \to \star$.

Оператор List отображает тип элементов в тип списка.

Вершина куба

Вершина куба — λC (Calculus of Constructions): все три зависимости сразу.

λC с индуктивными типами — основа Coq.

Полиморфизм: $\lambda 2$

В $\lambda \to$ терм монотипен (*monomorphic*): один фиксированный тип.

System F даёт один терм для всех типов сразу.

Определение 4 (System F ($\lambda 2$))

Термы зависят от типов.

Квантор $\forall \alpha$ связывает переменную типа.

Пример

Полиморфная тождественная функция:

$\Lambda \alpha . \lambda x : \alpha . x : \forall \alpha . \alpha \to \alpha .$

Зависимые типы: λP

Определение 5 (Зависимые типы)

Типы зависят от термов.

Тип $\mathrm{Vec}\ n$ — вектор длины n — меняется вместе со значением n.

Пример

Конкатенация векторов фиксирует длины в типе:

$\mathrm{concat} : \Pi n m : \mathrm{Nat}\ .\ \mathrm{Vec}\ n \to \mathrm{Vec}\ m \to \mathrm{Vec}\ (n + m)$.

Тип $\mathrm{Even}\ n$ населён ровно тогда, когда n чётно.

Зависимый тип — не булева функция, а утверждение о значении.

Пруф-ассистенты

Ассистенты доказательств превращают изоморфизм Карри–Ховарда в инструмент верификации.

Доказательство — это терм, а проверка доказательства — это проверка типа.

Пример

- Coq: корректность компилятора CompCert, теорема о четырёх красках.
- Lean 4: формализация современной математики.
- Agda: язык программирования и пруф-ассистент одновременно.

Принцип де Брёйна: ядро проверки типов — маленькая программа.

Если ядро приняло терм, доказательство корректно, как бы оно ни было получено.

Строгость и выразительность

Чем строже система типов, тем больше ошибок отлавливается на этапе компиляции.

Тем труднее выразить некоторые корректные программы.

Современные языки (Haskell, Rust, OCaml) ищут баланс: достаточно богатые типы для рекурсии и полиморфизма, достаточно строгие — чтобы исключить целые классы ошибок.

Логическая цена строгости: рекурсия, Y-комбинатор, неподвижные точки требуют выхода за пределы $\lambda \to$.

Полиморфизма System F недостаточно — нужны рекурсивные типы или оператор неподвижной точки.

Что мы поняли?

Дека началась с наблюдения: стрелка в типе устроена как импликация, и на этом совпадении держится всё остальное. Правила типизации воспроизводят логический вывод, нормализация превращается в гарантию остановки, а Карри–Ховард делает совпадение точным соответствием. Лямбда-куб показывает, куда расширяется соответствие: каждая новая зависимость термов и типов — новая логическая связка.

- Тип — инвариант о поведении терма. Функциональный тип $\sigma \to \tau$ структурно совпадает с импликацией.
- Типизируемость — существование дерева вывода. Сильная нормализация даёт гарантию остановки.
- Карри–Ховард: терм — доказательство, тип — высказывание, проверка типов — проверка доказательства.
- Зависимые типы типизируют утверждения о значениях. Пруф-ассистенты превращают изоморфизм в инструмент верификации.

Вопросы для самопроверки

Вопросы идут по ходу деки — от простых типов до зависимых типов. Ответ надёжнее, когда он построен: выведите тип, проведите редукцию, покажите отсутствие типа.

Вопросы для самопроверки — типы и доказательства

- Определите простые типы.
- Сформулируйте три правила типизации.
- Почему $\lambda x : \mathrm{Nat}\ .\lambda y : \mathrm{Bool}\ .x$ типизируется?
- Почему Y не типизируется?
- Что гарантирует сильная нормализация?
- Сформулируйте соответствие Карри–Ховарда.
- Какие логические связки дают тип-произведение, тип-сумма и пустой тип?
- Назовите три оси λ-куба Барендрегта.
- Какой тип у полиморфной тождественной функции?
- Почему тип $\mathrm{Vec}\ n$ невыразим в $\lambda\!\to$?

Что читать дальше

Термы без типов остались в деке о лямбда-исчислении: там — комбинаторы и нормальные формы без гарантии остановки. Почему доказательства в соответствии конструктивны — тема деки об интуиционизме. Инженерное продолжение изоморфизма — пруф-ассистенты — разобрано в деке о формальной верификации. В книге тема развёрнута в главе m28, рядом — главы m27 и m34.